

第10章 泛型与集合框架

建议学时:6-8 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握集合框架的体系:Collection(List/Set/Queue)与 Map
- 理解泛型与类型擦除,与 C++ 模板的异同
- 掌握通配符与 PECS 原则,能读懂框架源码里的复杂泛型
- 掌握 ArrayList 与 LinkedList 的原理与选型
- 理解 HashMap 的原理:哈希、冲突、扩容,以及它为什么依赖 equals/hashCode
- 掌握 Comparable 与 Comparator 两种排序方式
- 掌握迭代与遍历删除的正确姿势(fail-fast)

💡 从 C 到 Java:集合 = 造好的轮子

C 程序员每次需要"动态数组"都要自己造:malloc + realloc + 拷贝,手写链表还要维护头指针。Java 的集合框架把这些轮子全部造好:ArrayList(动态数组)、LinkedList(链表)、HashMap(哈希表)、HashSet、TreeMap(红黑树)——对应 C 里"轮子厂"级别的工程。java.util.Arrays.asList、List.of 甚至直接给你"声明即集合"。

泛型是 Java 对 C++ 模板的"打折版"。C++ 模板在编译期为每种类型生成代码(模板实例化);Java 泛型在编译期检查类型、运行期擦除类型(**类型擦除**): ArrayList<String> 和 ArrayList<Integer> 在运行期是同一个类 ArrayList。好处:不用复制代码;代价:不能 new T()、不能 instanceof T、List<String> 与 List<Object> 不是父子关系(协变问题,引出 PECS)。

HashMap 是第6章的"回扣"。哈希表的桶定位靠 hashCode,桶内比较靠 equals——第6章让你把这两个方法写对,就是为了这一刻。HashMap 的原理(数组+链表/红黑树、负载因子、扩容)是面试常客,也是理解"为什么键类要不可变"的根源。

排序两条路。C 的 qsort 需要一个比较函数指针;Java 是 Comparable(类自己说"我怎么排序")与 Comparator(外部说"这次怎么排序")——后者正是 lambda 的最佳舞台(第4章),comparingInt 等方法引用把排序写成一行。

学习顺序:先认识集合家族(§10.1),泛型与擦除(§10.2-10.3),常用容器原理(§10.4-10.5),排序与遍历(§10.6-10.7)。

📦 前置知识自查

- C 的动态数组/链表手写经验;qsort 的比较函数指针
- 第6章:equals/hashCode 契约(本章的 HashMap 直接依赖)
- 第4章:lambda 与方法引用(Comparator 的现代写法)
- 第1章:数组;第9章:并发容器与普通容器的区别

🚀 本章学习路线

1. **第一阶段(约 2 小时):**§10.1-10.3 集合家族与泛型擦除,理解"擦除"的三个不能
2. **第二阶段(约 2 小时):**§10.4-10.5 容器原理,重点 HashMap 的"数组+链表"心智模型
3. **第三阶段(约 2 小时):**§10.6-10.7 排序与遍历,完成章末实验
4. **验收标准:**能说出擦除的三个限制;能画出 HashMap 结构;能写 PECS 解释;能完成单词统计

本章知识导图

- §10.1 集合框架总览:Collection / List / Set / Map
- §10.2 泛型基础:泛型类 / 泛型方法 / 类型擦除
- §10.3 通配符与 PECS:extends / super
- §10.4 ArrayList vs LinkedList
- §10.5 HashMap 原理:哈希 / 冲突 / 扩容
- §10.6 排序:Comparable / Comparator
- §10.7 迭代与 fail-fast

§10.1 集合框架总览

10.1.1 家族树与 C 对照

接口	特性	实现(常用)	C 的对应物
List	有序、可重复、按下标访问	ArrayList、LinkedList	动态数组、链表
Set	无序(按实现)、不可重复	HashSet、TreeSet	手写哈希集合
Queue/Deque	队列语义(先进先出)	ArrayDeque、LinkedList	环形队列、链表队列
Map	键值对、键不可重复	HashMap、TreeMap	哈希表、二叉搜索树

示例 10-1 集合基本操作

```

import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;

public class CollectionBasics {
    public static void main(String[] args) {
        // List:有序可重复
        List<String> names = new ArrayList<>();
        names.add("张三");
        names.add("李四");
        names.add("张三"); // 允许重复
        System.out.println(names.get(1)); // 李四(按下标)
        System.out.println(names); // [张三, 李四, 张三]

        // Set:去重
        Set<String> uniq = new HashSet<>(names);
        System.out.println(uniq); // [李四, 张三](顺序不保证)

        // Map:键值对
        Map<String, Integer> scores = new HashMap<>();
        scores.put("张三", 88);
        scores.put("李四", 76);
        System.out.println(scores.get("张三")); // 88
        System.out.println(scores.getOrDefault("王五", -1)); // -1:取不到给默认

        // 不可变集合:JDK9+,声明即定,不可增删
        List<Integer> fixed = List.of(1, 2, 3);
        // fixed.add(4); // 抛 UnsupportedOperationException
        System.out.println(fixed);
    }
}

```

⚠ 易错点 1:集合存的是引用,不是副本

把对象放进集合,放的是引用——之后修改对象字段,集合里的"内容"跟着变。C 里往数组塞结构体是复制,Java 是"登记地址"。用 final/不可变对象当集合元素,能避免一大批"集合内容莫名变化"的 bug(第14章 record 的用武之地)。

§10.2 泛型基础与类型擦除

10.2.1 泛型类与泛型方法

示例 10-2 泛型类与泛型方法

```

import java.util.List;

public class GenericDemo {
    // 泛型类:T 是类型参数,实例化时指定
    static class Box<T> {
        private T value;

        Box(T value) { this.value = value; }

        T get() { return value; }

        void set(T value) { this.value = value; }
    }

    // 泛型方法:类型参数在方法上,独立于类
    static <T> T firstOr(List<T> list, T def) {
        return list.isEmpty() ? def : list.get(0);
    }

    // 类型参数的数量不限
    static class Pair<K, V> {
        final K key;
        final V value;

        Pair(K key, V value) { this.key = key; this.value = value; }
    }

    public static void main(String[] args) {
        Box<String> sb = new Box<>("hi"); // 菱形 <>:类型推断
        Box<Integer> ib = new Box<>(42);
        System.out.println(sb.get() + " " + ib.get());

        System.out.println(firstOr(List.of(1, 2), -1)); // 1
        System.out.println(firstOr(List.of(), "默认")); // 默认

        Pair<String, Integer> p = new Pair<>("张三", 88);
        System.out.println(p.key + ":" + p.value);
    }
}

```

10.2.2 与 C++ 模板对照:类型擦除的代价

对比项	C++ 模板	Java 泛型
实现时机	编译期:每种类型实例化一份代码	编译期检查、运行期擦除:只有一份代码
<code>new T()</code>	允许	不允许(擦除后没有 T 的类型信息)
<code>instanceof T</code>	允许	不允许(同上)
<code>List<String></code> 与 <code>List<Integer></code>	不同类型	同一个类(擦除后都是 List)
编译期检查	实例化时检查	使用处检查(编译期)
基本类型	int 直接用	不支持,用 Integer(自动装箱)

★ 重点:擦除带来的三个"不能"与一个"要"

① 不能 `new T()`; ② 不能 `instanceof T`; ③ 不能建泛型数组 `new T[]` (要建就用 `List<T>`)。一个"要":运行时拿类型信息,泛型类要传 `Class<T>` 参数(工厂模式: `create(Class<T> c) { c.getDeclaredConstructor().newInstance(); }`)。理解擦除,你就明白为什么 `List<String>` 不能直接转 `List<Object>` ——下一节的 PECS 就是为了对付这种"类型关系不直观"。

§10.3 通配符与 PECS

10.3.1 为什么需要 ? extends 与 ? super

因为擦除与不变性: `List<String>` 不是 `List<Object>` 的子类型,把 `List<String>` 传给"接收 `List<Object>`"的方法会编译失败。通配符解决"兼容多种具体类型"的需求: `? extends T` 表示"`T` 或 `T` 的子类",`? super T` 表示"`T` 或 `T` 的父类"。

示例 10-3 通配符与 PECS

```
import java.util.ArrayList;
import java.util.List;

public class WildcardDemo {
    // ? extends Number:可接收 List<Integer>/List<Double> 等
    static double sumOf(List<? extends Number> nums) {
        double s = 0;
        for (Number n : nums) {
            s += n.doubleValue();    // 读:安全,都是 Number
        }
        return s;
    }

    // ? super Integer:可接收 List<Integer>/List<Number>/List<Object>
    static void fillOnes(List<? super Integer> list, int count) {
        for (int i = 0; i < count; i++) {
            list.add(Integer.valueOf(i));    // 写:安全,Integer 是 Integer 的父类们的子类
        }
    }

    public static void main(String[] args) {
        System.out.println(sumOf(List.of(1, 2, 3)));    // 6.0
        System.out.println(sumOf(List.of(1.5, 2.5)));    // 4.0

        List<Number> nums = new ArrayList<>();
        fillOnes(nums, 3);
        System.out.println(nums);    // [0, 1, 2]
    }
}
```

★ 重点:PECS 口诀:Producer Extends, Consumer Super

方法只"产出"数据给调用方(读)→ 参数用 `? extends`;只"消费"数据(写/添加)→ 参数用 `? super`。读出写进:`extends` 的列表只能读(`add` 编译错误),`super` 的列表只能写(`get` 出来是 `Object`)。口诀来源: `Collections.copy(dest, src)` 的签名 `copy(List<? super T> dest, List<? extends T> src)` ——`dest` 被写所以 `super`,`src` 被读所以 `extends`。看到框架源码里这两个通配符,用口诀秒懂。

⚠ 易错点 2:泛型数组与 unchecked 警告

`List<String>[] arr = new List<String>[10]` 编译错误(不能建泛型数组); `new List[10]` 会给出 `unchecked` 警告。正确姿势:数组场景换 `List<List<String>>`。同理,老代码里裸类型(`List` 不加尖括号)会警告——生产代码把警告当错误对待,一律补齐类型参数。

§10.4 ArrayList vs LinkedList

操作	ArrayList(动态数组)	LinkedList(双向链表)
按下标访问 get(i)	O(1):直接算地址	O(n):从头/尾走一半
尾部添加 add(x)	O(1) 摊还(满了扩容,拷贝 1.5 倍)	O(1)
中间插入/删除	O(n):后续元素搬移	O(n) 找位置 + O(1) 改指针(定位仍是 O(n))
内存	连续数组,省(每个元素无额外指针)	每个节点 2 个指针 + 对象头,约贵 3-5 倍
C 对应	malloc + realloc 的动态数组	struct Node { data; next; prev; }

⚠ 易错点 3:LinkedList 的"头部操作"神话

面试常问"大量头插用 LinkedList 更快"——对,但只有 Queue/Deque 语义(头尾操作)时成立;大量"按下标访问"的场景 LinkedList 比 ArrayList 慢两个数量级。生产默认 List 就是 ArrayList;需要队列用 ArrayDeque;LinkedList 的适用面其实很窄。别被"链表万能"的刻板印象带偏。

§10.5 HashMap 原理

10.5.1 数组 + 链表(红黑树)的心智模型

HashMap 的存储:一个数组(桶),每个桶是一条链表(冲突时挂链),JDK8+ 链表超 8 个转红黑树。put 流程:算 hashCode → 与桶数取模定位 → 桶内 equals 逐个比,有则覆盖,无则插入。get 流程:同样定位后 equals 找目标。为什么键类要正确 equals/hashCode:定位靠 hash,比较靠 equals,哪个错了都找不到。

- **负载因子 0.75 与扩容**:元素数 > 桶数 × 0.75 时扩容为 2 倍并重哈希(全部重排,O(n) 的一次性代价)
- **默认容量 16**:预估数据量时用 `new HashMap<>(expectedSize)` 提前定容,减少扩容拷贝
- **允许 null**:HashMap 的键和值都允许 null;HashTable 不允许(老类,别用)
- **不保证顺序**:要有序用 LinkedHashMap(插入序)或 TreeMap(键序)

示例 10-4 HashMap 的键必须正确:反例演示

```

import java.util.HashMap;
import java.util.Map;

public class HashMapKeyDemo {
    // 错误键类:没重写 equals/hashCode(用 Object 的"引用相等")
    static class BadKey {
        final String id;
        BadKey(String id) { this.id = id; }
        // 忘了 equals/hashCode!
    }

    // 正确键类:第6章模板
    static class GoodKey {
        final String id;
        GoodKey(String id) { this.id = id; }

        @Override
        public boolean equals(Object o) {
            if (this == o) return true;
            if (!(o instanceof GoodKey)) return false;
            return id.equals(((GoodKey) o).id);
        }

        @Override
        public int hashCode() { return id.hashCode(); }
    }

    public static void main(String[] args) {
        Map<BadKey, String> badMap = new HashMap<>();
        badMap.put(new BadKey("s1"), "数据A");
        System.out.println("错误键类取回: "
            + badMap.get(new BadKey("s1"))); // null!找不着

        Map<GoodKey, String> goodMap = new HashMap<>();
        goodMap.put(new GoodKey("s1"), "数据B");
        System.out.println("正确键类取回: "
            + goodMap.get(new GoodKey("s1"))); // 数据B
    }
}

```

★ 重点:TreeMap 与排序语义

需要"按键有序遍历"时用 TreeMap(红黑树,按键的自然序或 Comparator)。它完全不需要 equals/hashCode,只靠 Comparable/Comparator 的 compareTo。选型:查得快 → HashMap;遍历要有序 → TreeMap;插入序保持 → LinkedHashMap。三者的 get 复杂度:HashMap O(1)、TreeMap O(log n)、LinkedHashMap O(1)。

§10.6 排序:Comparable 与 Comparator

10.6.1 两条路的区别

方式	语义	写法	适用
Comparable	类自己定义"自然顺序"	class X implements Comparable<X> { compareTo }	该类只有一个天然顺序(如学号)
Comparator	外部定义"本次顺序"	Comparator.comparingInt(...).reversed()	多种排序需求; 不侵入类定义

示例 10-5 两种排序实战

```

import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

public class SortDemo {
    // Comparable:学生自带自然顺序(按学号)
    static class Student implements Comparable<Student> {
        final String id;
        final String name;
        final int score;

        Student(String id, String name, int score) {
            this.id = id;
            this.name = name;
            this.score = score;
        }

        @Override
        public int compareTo(Student o) {
            return id.compareTo(o.id);          // 负数=我小,0=相等,正数=我大
        }

        @Override
        public String toString() { return id + ":" + name + "(" + score + ")"; }
    }

    public static void main(String[] args) {
        List<Student> list = new ArrayList<>(List.of(
            new Student("2024002", "李四", 76),
            new Student("2024001", "张三", 88),
            new Student("2024003", "王五", 93)));

        list.sort(null);                          // 用自然顺序(compareTo):按学号
        System.out.println("自然序: " + list);

        list.sort(Comparator.comparingInt(s -> s.score).reversed()); // 成绩降序
        System.out.println("成绩降序: " + list);

        list.sort(Comparator.comparing(s -> s.name));          // 姓名
        System.out.println("姓名序: " + list);

        // 链式:先成绩降序,同分按学号
        list.sort(Comparator.comparingInt((Student s) -> s.score)
            .reversed()
            .thenComparing(s -> s.id));
        System.out.println("多级排序: " + list);
    }
}

```



工程建议:排序写法规范

用 `Comparator.comparingInt/comparing`,不要手写 `(a, b) -> a.x - b.x`:减法在溢出时给出错误排序,double 的 NaN 更是灾难。需要自然顺序但类没实现 `Comparable` 时,写一个静态 `Comparator` 常量复用:`public static final Comparator<Student> BY_NAME = Comparator.comparing(s -> s.name);`。

§10.7 迭代与 fail-fast

10.7.1 遍历删除的正确姿势

用 `foreach`(或 `Iterator`)遍历集合时 直接调用 `list.remove(x)` 会抛 `ConcurrentModificationException(fail-fast)`——迭代器记账与集合实际修改次数不一致就报警。三种正确姿势:

示例 10-6 遍历删除三姿势


```

import java.util.ArrayList;
import java.util.List;
import java.util.ListIterator;

public class RemoveDemo {
    public static void main(String[] args) {
        // 需求:删除所有小于 60 的成绩
        List<Integer> scores = new ArrayList<>(List.of(45, 88, 62, 91, 55, 73));

        // 姿势 1:Iterator 的 remove(经典)
        var it = scores.iterator();
        while (it.hasNext()) {
            if (it.next() < 60) {
                it.remove();          // 迭代器的 remove 是安全的
            }
        }
        System.out.println("姿势1: " + scores);    // [88, 62, 91, 73]

        // 姿势 2:ListIterator 遍历中改(带下标)
        scores.addAll(List.of(45, 55));
        ListIterator<Integer> li = scores.listIterator();
        while (li.hasNext()) {
            if (li.next() < 60) {
                li.remove();
            }
        }
        System.out.println("姿势2: " + scores);    // [88, 62, 91, 73]

        // 姿势 3:Stream 过滤重建(函数式,推荐)
        scores.addAll(List.of(45, 55));
        scores = scores.stream().filter(s -> s >= 60).toList();
        System.out.println("姿势3: " + scores);    // [88, 62, 91, 73]
    }
}

```

⚠ 易错点 4:foreach 里 remove 的经典崩溃

`for (Integer s : scores) { if (s < 60) scores.remove(s); }` 运行时抛 `ConcurrentModificationException`; 更隐蔽的是“删最后一个元素恰好不报错”——行为依赖数据,灾难。记忆:foreach 底层就是 `Iterator`,修改必须通过 `Iterator.remove`;多线程修改同一集合也一样抛异常(这正是 fail-fast 的设计目的:尽早暴露,别静默错下去)。

本章速查表

主题	要点
家族树	List(有序可重)/ Set(去重)/ Queue(FIFO)/ Map(键值);实现:ArrayList/HashSet/ArrayDeque/HashMap
泛型	编译期检查、运行期擦除;Box<T> 与 static <T> 方法;菱形 <> 推断
擦除三不能	不能 new T() / instanceof T / 泛型数组;类型信息用 Class<T> 传参
通配符	? extends T(读)/ ? super T(写);List<String> 不是 List<Object>
PECS	Producer Extends, Consumer Super;copy(dest super, src extends)
ArrayList	动态数组:get O(1)、尾插摊还 O(1)、中间插删 O(n);默认选择
LinkedList	双向链表:头尾 O(1);按下标访问 O(n);适用面窄
HashMap	数组+链表(树);put/get 先 hash 定位后 equals 比较;负载因子 0.75 扩容 2 倍
键类铁律	键必须正确 equals/hashCode 且不可变;预估容量 new HashMap<>(n)
TreeMap	红黑树,按键序;只依赖 Comparable/Comparator,不依赖 hashCode
排序	Comparable(自然序)/ Comparator(外部序);用 comparingInt 不手写减法
遍历删除	Iterator.remove / ListIterator / Stream 过滤重建;foreach 里 remove 抛 ConcurrentModificationException
不可变集合	List.of / Set.of / Map.of;增删抛 UnsupportedOperationException

最小必会清单

- 能画出集合框架树,说出四种接口的语义与常用实现
- 能说出类型擦除与 C++ 模板的三个区别、三个"不能"
- 能写出 PECS 的例子并解释 copy 的签名
- 能说出 ArrayList 与 LinkedList 各操作复杂度,并说出默认选择
- 能画出 HashMap 的"数组+链表"结构,解释 put/get 流程
- 能解释为什么键类必须正确 equals/hashCode 且不可变
- 能写出三种遍历删除姿势,解释 fail-fast

自测题

Q 1. 为什么 `List<String>` 不能传给参数为 `List<Object>` 的方法?那传什么可以?

✅ 参考答案

若允许,方法里可以 add 一个 Integer 到该列表,破坏类型安全——所以泛型是不变的(不是协变)。接收方想兼容多种具体类型,参数应声明为 `List<? extends Object>` (或 `List<?>`)——只读兼容;如果要写,声明 `List<? super String>`。

Q 2. 泛型方法 `static <T> T max(List<T> list)` 为什么编译不过?怎么改?

✅ 参考答案

T 是任意类型,编译器无法保证有比较能力。改为 `static <T extends Comparable<? super T>> T max(List<T> list)`:约束 T 必须实现了 Comparable(其父类实现也可,所以是 ? super T)。这个签名正是 Collections.max 的原型——PECS 与泛型约束的综合应用。

Q 3. 往 HashMap 放 100 万个元素,性能明显下降。可能原因?怎么处理?

✓ 参考答案

① 键的 hashCode 分布差(如都取模到同一桶),链表变长——查键类的 hashCode 实现;② 扩容次数多——预估容量 `new HashMap<>(1_000_000 / 0.75)` 或更大,减少 rehash;③ 键类 equals 昂贵——检查实现。诊断用 `jvisualvm` 看对象分布,先查键类再调容量。

Q 4. 遍历 List 时想删除"倒数第 2 个元素",用哪种方式最稳?

✓ 参考答案

ListIterator 最稳:定位到最后,previous 一步,再 remove——迭代器的 remove 不会触发 fail-fast,而且能精确控制位置。注意 ListIterator 的 remove 删除的是"最近一次 next/previous 返回的元素",调用前必须先走一步。

Q 5. 自定义类放进 TreeSet,却不实现 Comparable,会发生什么?

✓ 参考答案

TreeSet 构造时若未传 Comparator,元素必须实现 Comparable,否则运行期抛 ClassCastException(不能转 Comparable)。对比:HashSet 只要求 equals/hashCode。所以"放 TreeSet 的类实现 Comparable,放 HashSet 的类实现 equals/hashCode"——两个体系各自为政。

Q 6. 写出代码:从单词列表统计词频,按出现次数降序输出前 3 个。

✓ 参考答案

```
List<String> words = List.of("a", "b", "a", "c", "a", "b", "d", "e", "f");
Map<String, Long> freq = new HashMap<>();
for (String w : words) {
    freq.merge(w, 1L, Long::sum);    // 没有则 1,有则加 1(方法引用)
}
freq.entrySet().stream()
    .sorted(Map.Entry.<String, Long>comparingByValue().reversed())
    .limit(3)
    .forEach(e -> System.out.println(e.getKey() + ": " + e.getValue()));
// a: 3,b: 2,c: 1(按次数降序前 3)
```

要点:merge 是"统计词频"的标准 API(比 get/put 三步安全);排序用 comparingByValue;Map.Entry.<String, Long> 是显式类型见证,让链式比较器编译通过。

章末实验题:单词统计系统

实验 10:单词统计系统

实验目标:编写一个文本单词统计系统,综合运用本章全部知识点:集合家族、泛型方法与泛型类、PECS、ArrayList/LinkedList 选型、HashMap 词频、Comparable/Comparator 排序、迭代与 fail-fast。

实验要求:

- 任务 1:读入文本文件(UTF-8),按空白与非字母切分单词(覆盖第7章 IO 与本章集合)
- 任务 2:用 `HashMap<String, Integer>` 统计词频,用 `merge` 或 `get/put`(覆盖 §10.5 HashMap)
- 任务 3:定义泛型方法 `<K, V extends Comparable<? super V>> List<Map.Entry<K, V>> sortByValue(Map<K, V> map, boolean desc)`,供词频排序复用(覆盖 §10.2-10.3 泛型与约束)
- 任务 4:定义一个词条类 `WordCount` implements `Comparable`(按次数降序、同次按字母),演示自然顺序(覆盖 §10.6 Comparable)
- 任务 5:用 `Comparator` 链式排序实现"次数降序 + 字母升序",与任务 4 对比(覆盖 §10.6 Comparator)
- 任务 6:遍历删除"出现次数 ≤ 1 "的词条,分别用 `Iterator.remove` 与 `Stream` 过滤两种方式,并各写一份(覆盖 §10.7 遍历删除)
- 任务 7:输出统计报告:总词数、去重词数、Top 10、删除低频词后的列表大小(覆盖全部)

实验环境:JDK 17+;准备一个英文文本文件 `article.txt`(几百词即可)。

第一步 完成读文件 + 分词 + 词频,打印 HashMap 内容验证

第二步 务 3 的泛型方法签名是本章难点:先写死类型跑通,再泛化

第三步 务 6 的两种删除方式输出应一致,可作为对照检查

参考答案要点

```

import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashMap;
import java.util.Iterator;
import java.util.List;
import java.util.Map;

/** 单词统计系统 — 第10章综合实验参考答案
 * 覆盖:集合家族、泛型方法、Comparable/Comparator、HashMap、遍历删除、IO
 */
public class WordStat {

    // 词条类:实现 Comparable(次数降序,同次按字母升序)
    static class WordCount implements Comparable<WordCount> {
        final String word;
        final int count;

        WordCount(String word, int count) {
            this.word = word;
            this.count = count;
        }

        @Override
        public int compareTo(WordCount o) {
            int byCount = Integer.compare(o.count, count); // 次数降序
            return byCount != 0 ? byCount : word.compareTo(o.word);
        }

        @Override
        public String toString() { return word + "(" + count + ")"; }
    }

    // 任务 3:泛型方法—按键值排序,支持升降序
    static <K, V extends Comparable<? super V>> List<Map.Entry<K, V>>
    sortByValue(Map<K, V> map, boolean desc) {
        List<Map.Entry<K, V>> entries = new ArrayList<>(map.entrySet());
        Comparator<Map.Entry<K, V>> cmp = Map.Entry.comparingByValue();
        entries.sort(desc ? cmp.reversed() : cmp);
        return entries;
    }

    public static void main(String[] args) throws IOException {
        // 任务 1:读文件 + 分词(只留字母,转小写)
        String text = Files.readString(Path.of("article.txt"),
            StandardCharsets.UTF_8).toLowerCase();
        String[] tokens = text.split("[^a-z]+");

        // 任务 2:词频统计(merge 标准姿势)
        Map<String, Integer> freq = new HashMap<>();
        for (String t : tokens) {
            if (!t.isEmpty()) {
                freq.merge(t, 1, Integer::sum);
            }
        }
        System.out.println("总词数: " + tokens.length
            + ", 去重词数: " + freq.size());

        // 任务 4:Comparable 自然顺序(直接转 WordCount 排序)
        List<WordCount> byNatural = new ArrayList<>();
        freq.forEach((w, c) -> byNatural.add(new WordCount(w, c)));
        byNatural.sort(null); // 用 compareTo
        System.out.println("Comparable 前 5: " + byNatural.subList(0, 5));

        // 任务 5:Comparator 链式排序(不侵入类的另一种顺序)
    }
}

```

```

List<WordCount> byComp = new ArrayList<>(byNatural);
byComp.sort(Comparator.comparingInt((WordCount wc) -> wc.count)
    .reversed()
    .thenComparing(wc -> wc.word)); // 次数降序 + 字母升序
System.out.println("Comparator 前 5: " + byComp.subList(0, 5));

// 任务 6a:Iterator.remove 删除低频词(次数 ≤ 1)
Iterator<Map.Entry<String, Integer>> it = freq.entrySet().iterator();
int removed1 = 0;
while (it.hasNext()) {
    if (it.next().getValue() <= 1) {
        it.remove();
        removed1++;
    }
}
System.out.println("Iterator 删除低频词: " + removed1
    + ", 剩余: " + freq.size());

// 任务 6b:Stream 过滤重建(不修改原集合,函数式)
Map<String, Integer> freq2 = new HashMap<>();
for (String t : tokens) {
    if (!t.isEmpty()) {
        freq2.merge(t, 1, Integer::sum);
    }
}
List<Map.Entry<String, Integer>> kept = freq2.entrySet().stream()
    .filter(e -> e.getValue() > 1)
    .toList();
System.out.println("Stream 过滤低频词: 保留 " + kept.size() + " 个");

// 任务 3:泛型方法复用(词频 Map 排序,降序前 10)
List<Map.Entry<String, Integer>> top = sortByValue(freq, true);
System.out.println("Top 10:");
top.stream().limit(10)
    .forEach(e -> System.out.println("  " + e.getKey()
        + ": " + e.getValue()));

// 任务 7:报告完成
System.out.println("统计完成,低频词(≤1)已清除。");
}
}

```

设计说明:任务 3 的签名是本章知识的高潮: `<K, V extends Comparable<? super V>>` 同时用到了泛型方法、泛型约束与 PECS(值类型 V 要可比较,其父类实现了 Comparable 也行);任务 4 与 5 故意让两种排序输出一致,证明"自然顺序"与"外部顺序"殊途同归;任务 6 的两种删除方式互相对照,验证 fail-fast 的规避。扩展方向:用 TreeMap 直接按字母序遍历输出全量词表;把 WordCount 改成 record(第14章)去掉样板代码;对超大文件用 Stream 流式统计(第4章)。

下一章预告:第11章 反射与动态特性。Class 对象、构造器与方法的反射调用、动态代理——框架的地基,和 Java 的"运行时自省"能力。